PAYMENT ORCHESTRATION LAYER WITH MULTI-GATEWAY FAILOVER

Category: Payment Systems Engineering | State Machine Design | API Architecture

Timeline: 15 Days | Full-Stack Back-End Development Project

Target: Computer Science / Software Engineering Students

TASK:

Design and implement a production-grade payment orchestration back-end that routes transactions across multiple payment gateways (Razorpay, Stripe, PayU, UPI) with intelligent routing based on success rates, cost, speed, and gateway health. The system must handle gateway failovers within 2 seconds, maintain idempotency across retries, implement webhook reconciliation, and store transaction state with complete audit trails. Students must design the routing algorithm, implement a state machine for the transaction lifecycle, and build a webhook ingestion pipeline with deduplication. The complete working codebase must be submitted as a GitHub repository transfer to ZethetaIntern.

PART A: TRAINING MATERIAL & LEARNING RESOURCES
PART B: GAMIFIED SIMULATION BUSINESS CONCEPT
PART C: CASE STUDIES & REAL-WORLD APPLICATIONS
PART D: 15-DAY PROJECT METHODOLOGY & DELIVERABLES
PART E: NO-CODE TOOLS & PLATFORMS GUIDE
PART F: SUBMISSION PROTOCOL

Important Note:

This Project is designed, developed, and administered solely by Zetheta Algorithms Private Limited ("Zetheta") for the purpose of assessing candidates, students, or event participants on behalf of engaging employers, institutes, or event organisers. Completion of this Project does not guarantee employment, academic outcomes, awards, or any other result - all such decisions remain at the sole discretion of the respective engaging party. Zetheta may use AI-assisted tools to evaluate submissions; such outputs are indicative in nature and may be inaccurate or incomplete. By participating, you consent to your performance data, scores, and submissions being shared with relevant engaging parties for assessment and decision-making purposes.

EXECUTIVE SUMMARY

This project positions students as the Back-End Architect at PayFlow Commerce, a fictional D2C e-commerce platform processing over 100,000 transactions daily across four payment gateways. The platform has been experiencing critical issues: gateway outages causing 12% of transactions to fail silently, double charges from unhandled webhook duplicates, and an inability to route transactions intelligently based on real-time gateway performance. The CEO has mandated a complete overhaul of the payment orchestration layer within 15 days.

The core engineering challenge students must solve is deceptively complex: How does a high-throughput payment system ensure that every single transaction either succeeds or fails cleanly, that no customer is ever double-charged, that gateway failures are detected and recovered from within 2 seconds, and that the entire transaction history is auditable to the rupee? The answer requires mastery of distributed systems patterns including idempotency, state machines, circuit breakers, webhook deduplication, and eventual consistency - concepts that separate production-grade payment engineers from basic CRUD developers.

Students will design and implement the complete payment orchestration layer including: a transaction state machine with 12+ explicit states and deterministic transitions; an intelligent gateway router that considers success rates, latency, cost, and health scores; a webhook ingestion pipeline that handles out-of-order delivery, duplicates, and signature verification; an idempotency layer that prevents double charges across retries and network failures; and a reconciliation engine that detects discrepancies between internal records and gateway settlements. Every component must be backed by a PostgreSQL database with complete audit trails.

The Payment Operations Board - a simulated review panel - will test each student's implementation against 15 pre-defined failure scenarios including gateway timeouts during capture, partial refunds with split settlements, webhook replay attacks, concurrent duplicate submissions, and cascade failures across multiple gateways. Students must demonstrate that their system handles every scenario without data corruption, without silent failures, and without manual intervention.

PART A: TRAINING MATERIAL & LEARNING RESOURCES

SECTION A1: PAYMENT GATEWAY ARCHITECTURE FUNDAMENTALS

A payment gateway is a technology intermediary that facilitates the transfer of information between a payment portal (such as a website or mobile application) and the acquiring bank or front-end processor. Understanding the internal architecture of payment gateways is essential before designing an orchestration layer that sits above them.

A1.1 The Payment Processing Chain

Every digital payment traverses a chain of participants. When a customer initiates a payment on an e-commerce platform, the transaction flows through the merchant's back-end to the payment gateway, which routes it to the appropriate payment network (card network, UPI switch, or net banking intermediary). The payment network communicates with the issuing bank (the customer's bank) to authorise the transaction. The response flows back through the same chain.

Understanding this chain is critical because failures can occur at any point, and the orchestration layer must handle each failure type differently:

Failure Point	Example Failure	Orchestrator Response	Recovery Time
Merchant to Gateway	Network timeout, DNS failure	Immediate failover to alternate gateway	< 500ms
Gateway to Network	Gateway internal error (5xx)	Retry with exponential backoff, then failover	1-2 seconds
Network to Issuer	Issuer bank downtime	Return 'issuer unavailable' - no retry	N/A (user retry)
Authorisation	Insufficient funds, fraud block	Return decline reason - no retry	N/A
Capture after Auth	Auth expired, amount mismatch	Void authorisation, re-initiate	2-5 seconds
Settlement	Reconciliation mismatch	Flag for manual review	Async (batch)

A1.2 Two-Phase Payment Flow - Authorise and Capture

Most payment gateways support a two-phase payment flow. In the first phase (authorisation), the gateway verifies that the customer has sufficient funds and places a hold on the amount. In the second phase (capture), the merchant confirms the transaction and the funds are transferred. This separation is critical for e-commerce because it allows merchants to authorise at checkout but capture only after the order is fulfilled.

The orchestration layer must track both phases independently. A transaction that is authorised but not captured within the gateway's hold period (typically 5–7 days) will have the hold released automatically, and the orchestrator must detect this and update the transaction state accordingly. Similarly, a partial capture (capturing less than the authorised amount) must be tracked with the remaining hold amount available for additional captures or void.

Key Implementation Detail: Your state machine must distinguish between **AUTHORISED**, **CAPTURE_INITIATED**, **CAPTURED**, **PARTIALLY_CAPTURED**, and **CAPTURE_FAILED** as separate states. Conflating any of these will lead to financial discrepancies.

A1.3 Gateway-Specific Behaviours

Each payment gateway has unique behaviours that the orchestration layer must account for. The following table summarises the critical differences between the four gateways used in this project:

Parameter	Razorpay	Stripe	PayU	UPI (NPCI)
Auth + Capture	Supported	Supported	Limited	N/A (instant)
Webhook Delivery	At-least-once	At-least-once	At-least-once	Callback-based
Webhook Signature	HMAC-SHA256	HMAC-SHA256 (Stripe-Signature header)	HMAC-SHA512	Digital signature (NPCI cert)
Idempotency Support	Client-generated key	Idempotency-Key header	txnid parameter	UPI txn ref number
Retry Behaviour	3 retries over 24 hrs	Up to 3 days	Best effort	Single callback
Settlement Cycle	T+2 business days	T+2 calendar days	T+3 business days	T+0 to T+1
Partial Refund	Supported	Supported	Supported	Not supported
Timeout (Auth)	30 seconds	30 seconds	45 seconds	60 seconds
Rate Limits	200 req/sec	100 req/sec (test), 10000 (live)	150 req/sec	N/A
Currency	INR only	Multi-currency	INR only	INR only

SECTION A2: TRANSACTION STATE MACHINE DESIGN

A state machine (specifically, a finite-state machine or FSM) is a computational model that defines a set of states, a set of events (transitions), and the rules governing which events cause transitions between which states. In payment processing, the state machine is the single source of truth for where a transaction is in its lifecycle.

A2.1 Why State Machines Are Non-Negotiable in Payments

Consider a naive implementation where transaction status is stored as a simple string column that can be updated to any value at any time. A bug in the refund handler could set a transaction from CREATED directly to REFUNDED, skipping the CAPTURED state entirely. The financial records would show a refund for a payment that was never captured - an irreconcilable discrepancy.

A properly implemented state machine prevents this by enforcing that transitions can only occur between explicitly defined state pairs. The **REFUND_INITIATED** event can only fire from the **CAPTURED** or **PARTIALLY_CAPTURED** states. Any attempt to transition from **CREATED** to **REFUNDED** would throw a InvalidStateTransitionException, preventing the corruption before it reaches the database.

A2.2 Required Transaction States (Minimum 12)

Your implementation must support at minimum the following 12 states. You may add additional states if your design requires them, but you must justify each addition in your architecture document:

State	Description	Valid Incoming Events	Valid Outgoing Events
CREATED	Transaction record initialised, no gateway interaction yet	New payment request	ROUTE_SELECTED, ABANDONED
ROUTE_SELECTED	Gateway chosen by routing algorithm	Router decision	AUTH_INITIATED, ROUTE_FAILED
AUTH_INITIATED	Authorisation request sent to gateway	Gateway API call made	AUTHORISED, AUTH_FAILED, AUTH_TIMEOUT
AUTHORISED	Gateway confirmed authorisation hold	Gateway auth success callback/response	CAPTURE_INITIATED, VOID_INITIATED, AUTH_EXPIRED

AUTH_FAILED	Gateway declined the authorisation	Gateway decline response	ROUTE_SELECTED (retry), FAILED
CAPTURE_INITIATED	Capture request sent to gateway	Merchant capture trigger	CAPTURED, CAPTURE_FAILED, PARTIALLY_CAPTURED
CAPTURED	Funds successfully transferred	Gateway capture confirmation	REFUND_INITIATED, SETTLED
PARTIALLY_CAPTURED	Partial amount captured	Gateway partial capture confirmation	CAPTURE_INITIATED (remainder), REFUND_INITIATED, SETTLED
CAPTURE_FAILED	Capture attempt failed after auth	Gateway capture error	CAPTURE_INITIATED (retry), VOID_INITIATED
REFUND_INITIATED	Refund request submitted to gateway	Merchant refund trigger	REFUNDED, PARTIALLY_REFUNDED, REFUND_FAILED
REFUNDED	Full refund processed	Gateway refund confirmation	Terminal state
FAILED	Terminal failure — no recovery possible	Max retries exceeded or hard decline	Terminal state

Additional states you should consider: ABANDONED (user dropped off), VOID_INITIATED (voiding an authorisation), VOIDED (authorisation successfully voided), AUTH_EXPIRED (hold period elapsed), SETTLED (settlement confirmed by gateway), PARTIALLY_REFUNDED (partial refund processed), REFUND_FAILED (refund attempt failed), DISPUTE_OPENED (chargeback initiated), DISPUTE_RESOLVED (chargeback resolved).

A2.3 State Transition Audit Trail

Every state transition must be logged to an immutable audit trail table. This is not optional — it is a regulatory requirement under PCI-DSS and RBI guidelines for payment intermediaries. The audit trail must capture:

Field	Type	Description	Example
id	UUID (PK)	Unique identifier for this audit entry	a1b2c3d4-...
transaction_id	UUID (FK)	Reference to the parent transaction	e5f6g7h8-...
from_state	ENUM	State before transition	AUTH_INITIATED
to_state	ENUM	State after transition	AUTHORISED
event	VARCHAR(100)	Event that triggered the transition	GATEWAY_AUTH_SUCCESS
gateway_reference	VARCHAR(255)	Gateway's transaction/order ID	pay_L1a2b3c4d5e6
gateway_response	JSONB	Full gateway response payload (PII redacted)	{"status":"captured",...}
metadata	JSONB	Additional context (IP, user agent, etc.)	{"ip":"103.x.x.x",...}
created_at	TIMESTAMPTZ	Timestamp of the transition	2025-03-19T10:30:00Z
created_by	VARCHAR(100)	System or user that triggered the transition	webhook_processor

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A3: INTELLIGENT GATEWAY ROUTING ALGORITHM

The routing algorithm is the brain of the payment orchestration layer. Given a payment request, it must select the optimal gateway based on multiple weighted factors, each of which changes in real time. This is not a simple round-robin or random selection - it is a multi-criteria decision problem with financial consequences.

A3.1 Routing Factors and Weights

Your routing algorithm must consider at minimum the following five factors. The weights are configurable and stored in the database (not hardcoded):

Factor	Weight (Default)	Data Source	Update Frequency	Description
Success Rate	35%	Internal transaction logs	Per-minute sliding window	Percentage of transactions that reached CAPTURED state in the last N minutes
Latency (P95)	20%	Internal response time tracking	Per-minute sliding window	95th percentile response time for auth requests in the last N minutes
Cost	20%	Gateway contract terms table	Updated on contract change	Transaction fee as percentage + fixed fee per transaction
Gateway Health	15%	Circuit breaker state + health check	Real-time	Binary (healthy/degraded/down) based on circuit breaker pattern
Payment Method Fit	10%	Gateway capability matrix	Static configuration	Whether the gateway supports the specific payment method (e.g., UPI is only via UPI gateway)

A3.2 Scoring Formula

The composite score for each gateway is calculated as follows. The gateway with the highest score is selected. If the selected gateway is in a degraded state (circuit breaker half-open), the second-highest scoring gateway is preferred unless the score difference is greater than 20%:

```
Score(gateway) =
(W_success * NormalizedSuccessRate(gateway)) +
(W_latency * (1 - NormalizedLatency(gateway))) +
(W_cost    * (1 - NormalizedCost(gateway))) +
(W_health  * HealthScore(gateway)) +
(W_fit     * FitScore(gateway, payment_method))
```

Where:
NormalizedSuccessRate = success_count / total_count (in sliding window)
NormalizedLatency = (p95_latency - min_latency) / (max_latency - min_latency)
NormalizedCost = (gateway_cost - min_cost) / (max_cost - min_cost)
HealthScore = 1.0 (healthy), 0.5 (degraded), 0.0 (down)
FitScore = 1.0 (supported), 0.0 (not supported)

A3.3 Circuit Breaker Pattern for Gateway Health

The circuit breaker pattern is essential for detecting and isolating failing gateways. Your implementation must follow the standard three-state model:

State	Behaviour	Transition Trigger
CLOSED (Normal)	All requests routed normally. Failure counter tracks consecutive failures.	Failure count exceeds threshold (e.g., 5 consecutive failures) → transition to OPEN
OPEN (Tripped)	No requests routed to this gateway. All requests fail-fast to alternative gateways.	Timeout period expires (e.g., 30 seconds) → transition to HALF-OPEN
HALF-OPEN (Testing)	Single test request allowed through. If it succeeds, circuit closes. If it fails, circuit re-opens.	Test request succeeds → CLOSED; Test request fails → OPEN (reset timeout)

Implementation Requirement: The circuit breaker must be implemented per-gateway and per-payment-method. A gateway may be healthy for card payments but degraded for UPI. Your circuit breaker configuration (failure threshold, timeout, half-open test count) must be stored in the database and be changeable without redeployment.

A3.4 Historical Gateway Performance Data

Your implementation will be tested against the following historical performance dataset. Use this data to seed your database and validate that your routing algorithm makes correct decisions:

Gateway	Hour	Success Rate	P95 Latency (ms)	Transactions	Cost (% + Fixed)
Razorpay	00:00–06:00	98.5%	320	4,200	2.0% + ₹2
Razorpay	06:00–12:00	97.2%	450	18,500	2.0% + ₹2
Razorpay	12:00–18:00	94.1%	780	32,000	2.0% + ₹2
Razorpay	18:00–24:00	96.8%	520	25,300	2.0% + ₹2
Stripe	00:00–06:00	99.1%	280	3,800	2.5% + ₹3
Stripe	06:00–12:00	98.8%	310	15,200	2.5% + ₹3
Stripe	12:00–18:00	97.5%	420	28,500	2.5% + ₹3
Stripe	18:00–24:00	98.2%	350	22,100	2.5% + ₹3
PayU	00:00–06:00	96.0%	400	2,100	1.8% + ₹1.5
PayU	06:00–12:00	93.5%	620	9,800	1.8% + ₹1.5
PayU	12:00–18:00	89.2%	950	15,500	1.8% + ₹1.5
PayU	18:00–24:00	91.8%	750	12,200	1.8% + ₹1.5
UPI	00:00–06:00	99.5%	180	5,600	0.0% + ₹0
UPI	06:00–12:00	99.2%	210	22,000	0.0% + ₹0
UPI	12:00–18:00	98.0%	350	38,000	0.0% + ₹0
UPI	18:00–24:00	98.8%	250	30,500	0.0% + ₹0

SECTION A4: IDEMPOTENCY IMPLEMENTATION PATTERNS

Idempotency is the property that performing an operation multiple times has the same effect as performing it once. In payment processing, idempotency is not a nice-to-have - it is the primary defence against double charges. Network failures, client timeouts, and retry storms can cause the same payment request to be submitted multiple times. Without idempotency, each submission creates a new charge.

A4.1 The Idempotency Key Strategy

Every payment API endpoint must accept an Idempotency-Key header (or request parameter). The key must be a client-generated UUID that uniquely identifies the payment intent. The server-side implementation works as follows:

```
FUNCTION process_payment(idempotency_key, payment_request):
 // Step 1: Check if this key has been seen before existing =
idempotency_store.get(idempotency_key)

IF existing IS NOT NULL: IF existing.status == "PROCESSING":
// Another request with same key is in-flight RETURN 409
Conflict ("Request already in progress")
ELSE:
// Return the cached response (same result as original) RETURN
existing.cached_response

// Step 2: Lock the key to prevent concurrent duplicates
lock_acquired = idempotency_store.insert
(key = idempotency_key, status = "PROCESSING", request_hash =
hash(payment_request),
expires_at = NOW() + 24 HOURS)

IF NOT lock_acquired:
// Race condition: another thread got the lock RETURN 409
Conflict

TRY:
// Step 3: Process the actual payment result =
gateway.charge(payment_request)

// Step 4: Cache the response idempotency_store.update
(key = idempotency_key, status = "COMPLETED", cached_response
= result)
```

```
RETURN result CATCH error:
// Step 5: On failure, mark key as failed (allow retry)
idempotency_store.update
(key = idempotency_key, status = "FAILED")
RAISE error
```

A4.2 Database Schema for Idempotency

Your idempotency store must be backed by a PostgreSQL table with the following minimum schema:

```
CREATE TABLE idempotency_keys (key VARCHAR(255) PRIMARY KEY,
request_hash    VARCHAR(64)  NOT NULL,  -- SHA-256 of request
body status VARCHAR(20)  NOT NULL DEFAULT 'PROCESSING' CHECK
(status IN ('PROCESSING', 'COMPLETED', 'FAILED')),
response_code   INTEGER,
response_body   JSONB,
created_at TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
updated_at TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
expires_at TIMESTAMPTZ  NOT NULL DEFAULT NOW() + INTERVAL '24
hours');

CREATE INDEX idx_idempotency_expires ON idempotency_keys
(expires_at)
WHERE status != 'COMPLETED';

-- Background job cleans up expired keys
-- DELETE FROM idempotency_keys WHERE expires_at < NOW();
```

ADD EXPERIENCE TO KNOWLEDGE

SECTION A5: WEBHOOK INGESTION & RECONCILIATION PIPELINE

Payment gateways communicate transaction status updates via webhooks - HTTP POST requests sent to a merchant-configured URL. Webhooks are the authoritative source of truth for transaction outcomes because they represent what the gateway's system actually processed, as opposed to what the merchant's system thinks happened.

A5.1 Webhook Delivery Guarantees

All four gateways in this project use an at-least-once delivery model. This means:

- The same webhook event may be delivered multiple times (duplicates)
- Webhooks may arrive out of order (event B before event A, even though A happened first)
- Webhooks may be delayed by minutes or even hours during gateway outages
- Webhooks may never arrive at all (rare, but possible - hence the need for reconciliation polling)

A5.2 Webhook Processing Pipeline Architecture

Your webhook processing pipeline must follow this architecture to handle the above guarantees:

```
[Gateway Webhook]
→ [Signature Verification Layer]
→ [Deduplication Layer (check event_id in processed_events
table)]
→ [Event Queue (persistent, e.g., PostgreSQL LISTEN/NOTIFY or
Redis Streams)]
→ [Event Processor (applies state machine transitions)]
→ [Audit Logger]
→ [Notification Dispatcher (optional)]
```

A5.3 Webhook Signature Verification

Every webhook must be verified before processing. An unverified webhook could be a spoofed request from an attacker attempting to mark fraudulent transactions as paid. Your implementation must verify signatures for all four gateways:

```
// Razorpay signature verification const crypto =
require('crypto'); function verifyRazorpayWebhook(body,
signature, secret) {const expectedSignature = crypto
```

```
.createHmac('sha256', secret)
.update(JSON.stringify(body))
.digest('hex');
return crypto.timingSafeEqual(
Buffer.from(signature),
Buffer.from(expectedSignature));}

// CRITICAL: Use timingSafeEqual, NOT ===
// String comparison (===) is vulnerable to timing attacks
// that can leak the expected signature byte-by-byte.
```

A5.4 Deduplication Strategy

Every gateway webhook contains a unique event identifier. Your deduplication layer must:

- Extract the event ID from the webhook payload (field name varies by gateway)
- Check if this event ID exists in the processed_events table
- If it exists, return HTTP 200 immediately (acknowledge but do not re-process)
- If it does not exist, insert it and proceed to processing
- The insert + processing must be atomic (same database transaction) to prevent race conditions between concurrent webhook deliveries

```
CREATE TABLE processed_webhook_events (event_id VARCHAR(255)
NOT NULL,
gateway VARCHAR(50)  NOT NULL,
event_type VARCHAR(100) NOT NULL,
payload_hash VARCHAR(64)  NOT NULL,
transaction_id UUID,
processed_at TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
PRIMARY KEY (gateway, event_id));

-- Composite PK ensures no two events with same ID from same
gateway
-- Different gateways may reuse event IDs, hence the composite
key
```

A5.5 Reconciliation Engine

The reconciliation engine is a batch process that runs periodically (recommended: every 15 minutes) to detect discrepancies between internal transaction records and gateway settlement data. It addresses the case where webhooks were lost or delayed beyond acceptable limits.

The reconciliation process:

- **Step 1 - Identify Stale Transactions:** Find transactions stuck in AUTH_INITIATED or CAPTURE_INITIATED for longer than the gateway's expected response time (configurable, default 5 minutes).
- **Step 2 - Poll Gateway Status:** For each stale transaction, call the gateway's 'fetch payment status' API using the gateway reference ID.
- **Step 3 - Reconcile:** Compare the gateway's reported status with the internal state. If they differ, apply the gateway's status as the source of truth and create a RECONCILIATION_OVERRIDE audit event.
- **Step 4 - Alert on Anomalies:** If a transaction is CAPTURED internally but the gateway reports it as FAILED, this is a critical anomaly requiring immediate human investigation. Log to a separate anomaly table and trigger an alert.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A6: DATABASE DESIGN FOR FINANCIAL DATA

Financial database design differs from typical application database design in several critical ways. The primary difference is immutability: financial records should never be updated in place. Instead, corrections are applied as new records that reference the original. This creates a complete, auditable history of every change.

A6.1 Core Database Schema

Your PostgreSQL database must include at minimum the following tables. You may add additional tables as needed, but these are non-negotiable:

Table	Purpose	Approximate Row Growth	Key Indexes
transactions	Core transaction records with current state	100K+ per day	PK, merchant_order_id, gateway_reference, state, created_at
transaction_state_log	Immutable audit trail of all state transitions	500K+ per day (avg 5 transitions per txn)	transaction_id, created_at, from_state, to_state
gateway_routes	Record of which gateway was selected and why	100K+ per day	transaction_id, gateway, score
idempotency_keys	Idempotency store for duplicate request detection	100K+ per day, pruned after 24h	PK (key), expires_at
processed_webhook_events	Deduplication store for webhook processing	200K+ per day (multiple events per txn)	PK (gateway, event_id), transaction_id
gateway_health_metrics	Per-minute gateway performance aggregates	5,760 per day (4 gateways x 1440 min)	gateway, recorded_at
reconciliation_log	Results of reconciliation batch runs	~100 per day	run_id, transaction_id, discrepancy_type
refunds	Refund records linked to parent transactions	~5K per day (5% refund rate)	transaction_id, gateway_refund_id, state

gateway_config	Gateway connection details and feature flags	~10 rows (static)	gateway_name
routing_config	Routing algorithm weights and thresholds	~5 rows (static)	config_key

A6.2 Data Type Discipline for Financial Amounts

This is where many junior developers make career-ending mistakes. Financial amounts must NEVER be stored as floating-point numbers (FLOAT, DOUBLE, REAL). Floating-point arithmetic introduces rounding errors that compound over thousands of transactions.

```
-- WRONG: This will cause rounding errors
amount FLOAT NOT NULL  -- DO NOT DO THIS

-- CORRECT: Option 1 - Use NUMERIC with fixed precision
amount NUMERIC(15, 2) NOT NULL  -- 15 digits total, 2 decimal
places
-- Supports up to 9,999,999,999,999.99

-- CORRECT: Option 2 - Store as integer paise/cents
amount_paise BIGINT NOT NULL  -- Store 100.50 as 10050
-- Application layer converts: display_amount = amount_paise /
100
-- This is the preferred approach for performance-critical
systems

-- CRITICAL: All amount operations must use the same
representation
-- NEVER mix NUMERIC and BIGINT amounts in the same
calculation
```

SECTION A7: RESTFUL API DESIGN FOR PAYMENT SYSTEMS

Your payment orchestration layer must expose a comprehensive REST API that enables the merchant application to initiate payments, check status, process refunds, and manage gateway configuration. The API must follow RESTful conventions, use proper HTTP methods and status codes, and be documented using OpenAPI 3.0 (Swagger) specification.

A7.1 Required API Endpoints (Minimum 20)

The following endpoints are the minimum required set. Your Swagger documentation must include request/response schemas, error codes, and example payloads for each:

#	Method	Endpoint	Description
1	POST	/api/v1/payments	Initiate a new payment
2	GET	/api/v1/payments/{id}	Retrieve payment details by ID
3	GET	/api/v1/payments?merchant_order_id={id}	Retrieve payment by merchant order ID
4	POST	/api/v1/payments/{id}/capture	Capture an authorised payment
5	POST	/api/v1/payments/{id}/void	Void an authorised payment
6	POST	/api/v1/payments/{id}/refund	Initiate a refund
7	GET	/api/v1/payments/{id}/refunds	List refunds for a payment
8	GET	/api/v1/payments/{id}/timeline	Get state transition history
9	POST	/api/v1/webhooks/razorpay	Razorpay webhook receiver
10	POST	/api/v1/webhooks/stripe	Stripe webhook receiver
11	POST	/api/v1/webhooks/payu	PayU webhook receiver
12	POST	/api/v1/webhooks/upi	UPI callback receiver
13	GET	/api/v1/gateways	List all configured gateways
14	GET	/api/v1/gateways/{name}/health	Get gateway health status
15	GET	/api/v1/gateways/{name}/metrics	Get gateway performance metrics
16	PUT	/api/v1/gateways/{name}/config	Update gateway configuration
17	GET	/api/v1/routing/config	Get current routing weights
18	PUT	/api/v1/routing/config	Update routing weights
19	POST	/api/v1/reconciliation/trigger	Manually trigger reconciliation run

20	GET	/api/v1/reconciliation/reports/{run_id}	Get reconciliation report
21	GET	/api/v1/analytics/success-rate	Gateway success rate analytics
22	GET	/api/v1/analytics/volume	Transaction volume analytics
23	GET	/api/v1/health	System health check endpoint

A7.2 Standard Error Response Format

All API errors must follow a consistent format. Do not return gateway-specific error messages directly - translate them into your standard error schema:

```
{"error": {"code": "PAYMENT_AUTH_FAILED", "message": "Payment
authorisation was declined by the issuing bank.","details":
{"gateway": "razorpay","gateway_error_code":
"BAD_REQUEST_ERROR","gateway_error_description": "The card
issuer has declined this transaction.","suggestion": "Please
try a different payment method or contact your
bank."},"request_id": "req_a1b2c3d4e5f6","timestamp":
"2025-03-19T10:30:00.000Z"}}
```

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION A8: CONCURRENCY & DISTRIBUTED LOCKING PATTERNS

Payment systems must handle concurrent requests safely. Two customers placing orders simultaneously, two webhooks arriving for the same transaction at the same instant, or a retry colliding with the original request - these are not edge cases; they are the normal operating condition of a system processing 100K+ transactions daily.

A8.1 Optimistic vs Pessimistic Locking

Your implementation must choose between optimistic and pessimistic locking for state machine transitions. The choice has significant implications for throughput and correctness:

Aspect	Optimistic Locking	Pessimistic Locking
Mechanism	Version column checked at UPDATE time. If version has changed since SELECT, the UPDATE fails.	SELECT FOR UPDATE acquires a row-level lock. Other transactions block until the lock is released.
Code Pattern	UPDATE transactions SET state = 'CAPTURED', version = version + 1 WHERE id = ? AND version = ?	BEGIN; SELECT * FROM transactions WHERE id = ? FOR UPDATE; -- process -- UPDATE ...; COMMIT;
Throughput	Higher: no locks held during processing. Only conflicts cause retries.	Lower: locks held for the duration of processing (including gateway calls).
Correctness Risk	Application must handle OptimisticLockException and retry the entire operation.	Guaranteed correctness as long as the lock is held for the minimal required duration.
Recommended For	High-read, low-write scenarios. State checks that don't involve external calls.	Critical sections where external calls (gateway APIs) are made between read and write.

Recommended Approach: Use pessimistic locking (SELECT FOR UPDATE) for the state machine transition itself, but release the lock before making gateway API calls. The pattern is: acquire lock → validate current state → update state to intermediate (e.g., AUTH_INITIATED) → release lock → call gateway → acquire lock again → update state to final (e.g., AUTHORISED or AUTH_FAILED) → release lock.

A8.2 Advisory Locks for Idempotency

PostgreSQL advisory locks provide application-level locking without being tied to specific rows or tables. They are ideal for the idempotency implementation because the lock is on the idempotency key itself (which may not yet exist as a row):

```
-- Acquire an advisory lock using the hash of the idempotency
key
-- pg_advisory_xact_lock is transaction-scoped (auto-released
on COMMIT/ROLLBACK)
SELECT pg_advisory_xact_lock(hashtext('idem_' || $1));

-- Now safe to check-and-insert the idempotency key
INSERT INTO idempotency_keys (key, request_hash, status)
VALUES ($1, $2, 'PROCESSING')
ON CONFLICT (key) DO NOTHING
RETURNING key;

-- If RETURNING is empty, the key already existed (duplicate
request)
-- If RETURNING has the key, we acquired the lock and can
proceed
```

A8.3 Dead Letter Queue Pattern

When webhook processing fails (database unavailable, unexpected payload format, application error), the webhook must not be lost. Implement a dead letter queue (DLQ) pattern:

- **Primary Queue:** Incoming webhooks are written to a processing queue (PostgreSQL table or Redis stream).
- **Processing:** A consumer reads from the queue, processes the webhook, and marks it as completed.
- **Failure Handling:** If processing fails after 3 retries with exponential backoff, the webhook is moved to the dead letter queue.
- **DLQ Monitoring:** A dashboard shows the DLQ depth. Any non-zero DLQ depth triggers an alert.
- **Manual Replay:** An admin API endpoint allows replaying individual items from the DLQ after the root cause is fixed.

```
CREATE TABLE webhook_queue (
    id              BIGSERIAL PRIMARY KEY,
    gateway         VARCHAR(50)  NOT NULL,
    event_id        VARCHAR(255) NOT NULL,
```

```
    payload          JSONB         NOT NULL,
    signature        TEXT          NOT NULL,
    status           VARCHAR(20)   NOT NULL DEFAULT 'PENDING'
                     CHECK (status IN
('PENDING','PROCESSING','COMPLETED','FAILED','DLQ')),
    retry_count      INTEGER       NOT NULL DEFAULT 0,
    max_retries      INTEGER       NOT NULL DEFAULT 3,
    next_retry_at    TIMESTAMPTZ,
    error_message    TEXT,
    created_at       TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
    processed_at     TIMESTAMPTZ
);

CREATE INDEX idx_webhook_queue_pending
    ON webhook_queue (next_retry_at)
    WHERE status IN ('PENDING', 'FAILED');
```

A8.4 Rate Limiting & Backpressure

Your orchestration layer must implement outbound rate limiting to respect each gateway's API limits. Exceeding rate limits leads to 429 responses, which count as failures and can trip circuit breakers - creating a self-inflicted outage.

Gateway	Rate Limit	Implementation
Razorpay	200 req/sec	Token bucket with 200 tokens, refilled at 200/sec. Requests beyond capacity are queued (not dropped).
Stripe	100 req/sec (test mode)	Respect Stripe's Retry-After header. Use exponential backoff with jitter.
PayU	150 req/sec	Sliding window counter. Requests exceeding the window are delayed until the next window.
UPI (NPCI)	Depends on aggregator	Follow aggregator-specific limits. Typically 50-100 req/sec per merchant.

Implementation Tip: Use a leaky bucket or token bucket algorithm. Store the bucket state in Redis (if available) or PostgreSQL. The rate limiter must be gateway-specific and must include a monitoring endpoint that shows current utilisation per gateway (e.g., "Razorpay: 145/200 req/sec").

A8.5 Distributed Tracing

For debugging production issues, your orchestration layer should generate a unique trace ID for each payment request and propagate it through all components:

- **Trace ID Generation:** Generate a UUID v4 trace ID when the payment request is received.
- **Propagation:** Include the trace ID in all gateway API calls (as a custom header), all database audit entries, all log messages, and all webhook processing.
- **Correlation:** When a webhook arrives, extract the trace ID from the original transaction record and use it for all subsequent log messages and state transitions.
- **Logging Format:** All log messages must be structured JSON with the trace ID as a top-level field. This enables searching for all events related to a single payment using a single query.

```
// Structured log format
{
  "timestamp": "2025-03-19T10:30:00.123Z",
  "level": "INFO",
  "trace_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "transaction_id": "txn_9876543210",
  "component": "gateway_router",
  "action": "route_selected",
  "gateway": "razorpay",
  "score": 0.87,
  "duration_ms": 12,
  "message": "Selected Razorpay with score 0.87 (success_rate=0.35, latency=0.18,
cost=0.16, health=0.13, fit=0.05)"
}
```

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

Note on the Simulation Concept: The gamified simulation concept presented in this section is provided as a reference framework. Students may choose to follow this concept or design their own original gamified simulation - with a different name, structure, narrative, or game mechanics - as long as it meets the assessment criteria outlined in Part F.

SECTION B1: SIMULATION CONTEXT - PAYFLOW COMMERCE

PayFlow Commerce is a fictional D2C (Direct-to-Consumer) e-commerce platform headquartered in Mumbai, India. The platform sells premium lifestyle products and processes an average of 100,000+ transactions daily across four payment gateways. You have been hired as the Back-End Architect tasked with rebuilding the payment orchestration layer from scratch.

B1.1 Company Profile

Parameter	Value
Company Name	PayFlow Commerce Private Limited
Industry	D2C E-Commerce (Lifestyle, Electronics, Fashion)
Monthly GMV	₹45 Crore (₹450 Million)
Average Order Value	₹2,800
Daily Transaction Volume	105,000 (avg); 180,000 (peak during sales)
Active Gateways	Razorpay, Stripe, PayU, UPI (via NPCI)
Payment Methods	Credit Card, Debit Card, UPI, Net Banking, Wallets
Current Success Rate	88.2% (target: 96%+)
Current Avg Latency	3.2 seconds (target: < 1.5 seconds)
Monthly Revenue Lost to Failures	₹5.3 Crore (~11.8% of GMV)

B1.2 Current Problems You Must Solve

The CTO's internal audit has identified the following critical issues. Your implementation must demonstrate solutions for each:

#	Problem	Impact	Root Cause	Expected Solution
P1	Silent transaction failures	4,200 transactions/day fail without customer notification	No failover mechanism; single gateway attempt	Multi-gateway routing with automatic failover
P2	Double charges	~180 customers/month are charged twice for the same order	No idempotency layer; webhook duplicates processed as new events	Idempotency key implementation + webhook deduplication
P3	Unreconciled transactions	₹12 Lakh/month in unreconciled discrepancies	No automated reconciliation; manual Excel-based matching	Automated reconciliation engine with anomaly detection
P4	Slow checkout during peak hours	32% cart abandonment rate during 6–9 PM	All traffic routed to single gateway regardless of load	Intelligent multi-criteria routing algorithm
P5	No audit trail	Unable to trace transaction history for dispute resolution	Status stored as single mutable column with no history	Immutable state machine with transition audit log
P6	Gateway outage blindness	Average 45 minutes to detect a gateway outage	No health monitoring; relies on customer complaints	Circuit breaker pattern with real-time health scoring

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION B2: THE 15 FAILURE SCENARIOS

The Payment Operations Board will test your implementation against the following 15 failure scenarios. Each scenario is designed to test a specific aspect of your system's resilience. Your code must handle all 15 without data corruption, without silent failures, and without manual intervention.

FS-01: Gateway Timeout During Authorisation [50 Points]

Scenario: Razorpay times out (no response for 30 seconds) during an auth request.

Expected System Behaviour: Circuit breaker detects timeout, increments failure counter. Router fails over to Stripe (next highest score). Transaction state moves from AUTH_INITIATED to ROUTE_SELECTED (new gateway). Total failover time < 2 seconds.

FS-02: Duplicate Webhook Delivery [50 Points]

Scenario: Razorpay sends the same payment.captured webhook 3 times within 10 seconds.

Expected System Behaviour: First webhook is processed normally (state → CAPTURED). Second and third webhooks are identified as duplicates via event_id in processed_webhook_events table. HTTP 200 returned for all three. No duplicate state transitions or financial records.

FS-03: Double Submit by Customer [50 Points]

Scenario: Customer clicks 'Pay' button twice in rapid succession, sending two identical payment requests with the same idempotency key.

Expected System Behaviour: First request acquires the idempotency lock and proceeds to gateway. Second request hits the idempotency check, finds status = PROCESSING, returns 409 Conflict. Only one gateway charge is created.

FS-04: Gateway Returns 5xx During Capture [50 Points]

Scenario: After successful authorisation on PayU, the capture request returns HTTP 502 Bad Gateway.

Expected System Behaviour: Orchestrator retries capture with exponential backoff (1s, 2s, 4s). After 3 failed retries, state moves to CAPTURE_FAILED. System polls gateway status API to determine if capture was actually processed server-side (late success pattern).

FS-05: Partial Capture with Remaining Hold [50 Points]

Scenario: Merchant captures ₹800 of a ₹1,200 authorised transaction (customer returned one item before shipping).

Expected System Behaviour: State transitions to PARTIALLY_CAPTURED. Remaining ₹400 hold is tracked. Merchant can either capture the remainder or void the remaining hold. Audit trail shows both the partial capture and the remaining holdable amount.

FS-06: Webhook Arrives Before API Response [50 Points]

Scenario: Stripe sends the payment_intent.succeeded webhook before the synchronous API response returns to the orchestrator.

Expected System Behaviour: Webhook processor checks transaction state. If still in AUTH_INITIATED (API response pending), webhook transitions state to CAPTURED. When the API response finally arrives, the state machine rejects the duplicate transition gracefully (no error, no double processing).

FS-07: Cascade Gateway Failure [75 Points]

Scenario: Razorpay goes down (circuit breaker trips). PayU starts degrading (80% success rate). Stripe is at capacity (rate limit 429 responses).

Expected System Behaviour: Router dynamically shifts all eligible traffic to UPI. Non-UPI transactions are queued with exponential backoff for Stripe (respecting Retry-After header). Circuit breaker on Razorpay prevents any traffic. PayU receives reduced traffic (proportional to degraded health score).

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

FS-08: Refund on Already-Settled Transaction [50 Points]

Scenario: Merchant initiates a refund for a transaction that was captured 3 days ago and has already been settled by the gateway.

Expected System Behaviour: System identifies the transaction is in SETTLED state. Refund is still valid (within refund window). Refund request is sent to gateway. Gateway processes refund against settled funds. State transitions: SETTLED → REFUND_INITIATED → REFUNDED.

FS-09: Concurrent Idempotency Race Condition [75 Points]

Scenario: Two servers in a load-balanced setup receive the same payment request simultaneously (same idempotency key), within microseconds of each other.

Expected System Behaviour: Database-level advisory lock or SELECT FOR UPDATE ensures only one request proceeds. The other receives 409 Conflict. No duplicate charges under any timing scenario. This must be demonstrated with a concurrent test.

FS-10: Webhook Replay Attack [50 Points]

Scenario: An attacker captures a legitimate Razorpay webhook and replays it with a modified amount (attempting to mark a ₹100 payment as ₹10,000).

Expected System Behaviour: Signature verification fails because the modified payload does not match the HMAC signature. Request is rejected with 401 Unauthorized. Security audit log records the attempt with the source IP.

FS-11: Reconciliation Detects Missing Settlement [75 Points]

Scenario: The reconciliation engine finds 5 transactions marked as CAPTURED internally but the gateway's settlement report shows them as failed/reversed.

Expected System Behaviour: Reconciliation creates ANOMALY records for each discrepancy. Transactions are flagged as RECONCILIATION_MISMATCH. Alert is dispatched. Automatic refund is NOT triggered (requires human review for settlement discrepancies).

FS-12: UPI Collect Flow Timeout [50 Points]

Scenario: A UPI collect request is initiated. The customer does not approve within the 5-minute UPI mandate window.

Expected System Behaviour: Orchestrator polls UPI status after the timeout window. UPI callback confirms EXPIRED status. State transitions: AUTH_INITIATED → AUTH_EXPIRED. Customer is notified. No retry is attempted (UPI collect cannot be force-retried).

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

FS-13: Gateway Idempotency Key Collision [50 Points]

Scenario: Two different merchants (multi-tenant scenario) accidentally generate the same idempotency key UUID.

Expected System Behaviour: Your idempotency key must be scoped to the merchant. The database key is a composite of (merchant_id, idempotency_key). Same idempotency key from different merchants are treated as different requests.

FS-14: Hot-Path Database Connection Exhaustion [75 Points]

Scenario: During a flash sale, the transaction volume spikes 5x. Database connection pool (max 20 connections) is exhausted.

Expected System Behaviour: System implements connection pooling with PgBouncer or equivalent. Long-running reconciliation queries are routed to a read replica. Webhook processing uses a queue to decouple ingestion from processing. Circuit breaker on internal resources (DB, cache) prevents cascade failure.

FS-15: State Machine Corruption Attempt [50 Points]

Scenario: A buggy refund handler attempts to transition a CREATED transaction directly to REFUNDED (skipping all intermediate states).

Expected System Behaviour: State machine rejects the invalid transition with an InvalidStateTransitionException. The attempted transition is logged in the audit trail as a REJECTED_TRANSITION event. The refund handler receives a clear error message indicating the valid transitions from CREATED state.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION B3: PERFORMANCE BENCHMARKS

Your implementation will be benchmarked against the following performance targets. These are measured using automated load testing (k6 or Artillery) against your Docker-deployed system:

Metric	Target	Measurement Method	Weight
Payment Initiation (P95)	< 500ms	Time from API request to gateway call initiated	20%
Failover Latency	< 2 seconds	Time from primary gateway failure detection to alternate gateway response	25%
Webhook Processing (P95)	< 200ms	Time from webhook receipt to state transition committed	15%
Concurrent Payment Handling	100 req/sec sustained for 5 minutes	k6 constant-arrival-rate test	20%
Idempotency Check	< 10ms	Time to detect and return cached response for duplicate request	10%
Reconciliation Run	< 30 seconds for 10,000 transactions	Time for full reconciliation batch	10%

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION B4: SIMULATION EXECUTION PROTOCOL

The Payment Operations Board simulation will be executed in a controlled environment. Understanding the execution protocol is essential for preparing your implementation.

B4.1 Testing Environment

Your Docker Compose setup will be deployed on a standard testing VM with the following specifications:

Resource	Specification
CPU	4 vCPU (Intel Xeon, 2.5 GHz)
Memory	8 GB RAM
Storage	50 GB SSD
Network	1 Gbps internal; no external internet access
OS	Ubuntu 22.04 LTS
Docker	Docker Engine 24.0+ with Docker Compose v2
PostgreSQL	Version 15 (run from your docker-compose)

Your application must start successfully within 60 seconds of running docker-compose up. If the application does not respond to the health check endpoint within 60 seconds, the automated test suite will abort and assign zero marks for the simulation.

B4.2 Test Execution Sequence

The automated test harness executes scenarios in the following order:

Phase	Duration	What Happens
Phase 1: Warm-Up	30 seconds	10 standard payment requests are sent to verify basic functionality. All must succeed.
Phase 2: Scenario Testing	~20 minutes	Each of the 15 failure scenarios is executed sequentially. The system is reset between scenarios.
Phase 3: Load Testing	5 minutes	100 concurrent payments per second for 5 minutes using k6.
Phase 4: Reconciliation	2 minutes	Reconciliation is triggered manually; accuracy is measured against known-good dataset.

Phase 5: Data Integrity Audit	5 minutes	Automated SQL queries verify: no orphaned records, no duplicate charges, all amounts reconcile, all state transitions are valid.

B4.3 Mock Gateway Behaviour

Since your implementation cannot connect to real payment gateways, you must implement mock gateways that simulate realistic behaviour. The automated test harness will send specific headers to trigger specific mock responses:

```
// Mock Gateway Control Headers
// These headers are sent by the test harness to control mock gateway responses

X-Mock-Response: success            // Return a successful auth/capture
X-Mock-Response: timeout       // Simulate a 30-second timeout (no response)
X-Mock-Response: server-error  // Return HTTP 502
X-Mock-Response:   decline                    //   Return   a declined/insufficient funds response
X-Mock-Response: rate-limit        // Return HTTP 429 with Retry-After header
X-Mock-Delay-Ms: 2000           // Add 2-second delay before responding
X-Mock-Gateway-Down: true         // Simulate gateway being completely unreachable
```

Your mock gateways must respect these headers so that the automated test harness can trigger specific failure conditions deterministically. Without this mock control interface, the test harness cannot execute failure scenarios.

SECTION B5: SCORING & LEADERBOARD

Student submissions are ranked on a leaderboard visible to all interns in the Back-End Developer cohort. The leaderboard is updated after the automated testing phase (Days 16-23) and displays:

Rank Metric	Weight in Leaderboard	Description
Scenario Pass Rate	40%	Number of failure scenarios passed out of 15
Code Quality Score	25%	Combined score from linting, coverage, and review
Performance Score	20%	How well performance benchmarks are met (B3 targets)
Architecture Score	15%	Documentation quality and design pattern usage

Top 3 Recognition: The top 3 students on the leaderboard will receive: a LinkedIn recommendation from the CTO, priority consideration for a full-time offer, and their architecture will be featured as a reference implementation for future cohorts.

Minimum Bar: Students scoring below 500/1000 total or passing fewer than 8 of the 15 failure scenarios will be required to resubmit within 7 days with corrections. A second failure results in a No-Go recommendation for the internship programme.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART C: CASE STUDIES & REAL-WORLD APPLICATIONS

These case studies are drawn from real-world payment system incidents (names and details modified). Each case study includes a scenario description, the actual impact, the root cause analysis, and the design pattern that would have prevented the incident. Students must reference these case studies in their architecture documentation.

SECTION C1: THE DIWALI SALE DOUBLE CHARGE DISASTER

C1.1 Background

During the 2024 Diwali sale, a major Indian e-commerce platform processed 2.8 million transactions in a 6-hour window. At peak load (approximately 18:00 IST), the Razorpay integration experienced intermittent timeouts lasting 15-45 seconds. The platform's payment handler, lacking idempotency, retried failed requests - creating duplicate charges.

C1.2 The Incident Timeline

Time	Event	Impact
17:45 IST	Razorpay latency increases from 400ms to 8 seconds	Timeout threshold (5s) starts triggering retries
17:52 IST	Retry storm begins - each timeout generates 3 retries	Gateway load increases 4x; latency worsens to 15s
18:05 IST	First customer complaints of double charges appear	Customer support team receives 340 complaints in 20 minutes
18:20 IST	Engineering team identifies the issue	Retries disabled manually via config flag
18:35 IST	Retries disabled; latency recovers to 600ms	Damage assessment begins
18:45 IST	Razorpay confirms 14,200 duplicate charges	₹3.2 Crore in erroneous charges
19:00–23:00 IST	Manual reconciliation and batch refund processing	Team of 12 engineers works through the night
+3 days	All refunds processed; customer communication completed	Estimated brand damage: ₹15 Crore in lost future revenue

C1.3 Root Cause Analysis

The root cause was a combination of three missing components: (1) No idempotency keys were sent with retry requests, so each retry created a new payment on the gateway side; (2) No circuit breaker was implemented, so increasing latency led to increasing retries, creating a positive feedback loop; (3) The retry logic was synchronous and blocked the checkout flow, compounding user frustration and leading to manual re-attempts by customers.

C1.4 Design Patterns That Would Have Prevented This

- **Idempotency Keys:** Each payment intent should have been tagged with a client-generated UUID. Retries with the same UUID would have been deduplicated on the gateway side, preventing duplicate charges entirely.
- **Circuit Breaker:** After 5 consecutive timeouts, the circuit breaker would have tripped, routing all new payments to Stripe or PayU within 500ms. The retry storm would never have materialised.
- **Exponential Backoff:** Instead of immediate retries, exponential backoff (1s, 2s, 4s) with jitter would have reduced the load on the already-stressed gateway.
- **Asynchronous Retry Queue:** Retries should have been enqueued to a persistent queue and processed asynchronously, decoupling the retry logic from the customer's checkout session.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION C2: THE SILENT SETTLEMENT LEAK

C2.1 Background

A subscription-based SaaS platform operating across India and Southeast Asia discovered a ₹8.7 Lakh monthly discrepancy between their internal revenue records and actual bank settlements. The issue had been ongoing for 11 months before detection, resulting in a cumulative loss of approximately ₹95 Lakh.

C2.2 How It Went Undetected

The platform's payment system marked transactions as CAPTURED when the gateway returned a synchronous success response. However, for approximately 1.2% of transactions, the gateway later reversed the capture during settlement (reasons included: fraud detection, issuer disputes, and technical reversals). The platform had no mechanism to receive or process these post-capture settlement webhooks.

The monthly discrepancy of ₹8.7 Lakh represented approximately 1.2% of their ₹7.25 Crore monthly processing volume - a small enough percentage to be attributed to "normal variance" by the finance team. It was only discovered during an annual audit when the cumulative amount became significant.

C2.3 Technical Failure Analysis

- **Webhook Configuration Gap:** The platform had configured webhooks for payment.captured events but had not subscribed to payment.reversed, payment.failed, and dispute.created events.
- **No Reconciliation Engine:** There was no automated process to compare internal records against gateway settlement reports.
- **Single-Status Architecture:** The transaction table had a single status column that was updated in place. Once set to CAPTURED, there was no mechanism to move it to REVERSED or DISPUTED.
- **No Settlement Tracking:** The system did not track which transactions had been included in which settlement batch, making batch-level reconciliation impossible.

C2.4 Lessons for Your Implementation

Your reconciliation engine (tested in FS-11) must: subscribe to ALL webhook event types (not just success events); run periodic reconciliation against gateway settlement APIs; track settlement batch inclusion for every captured transaction; and flag discrepancies immediately rather than relying on periodic financial audits.

SECTION C3: THE UPI MANDATE EXPLOITATION

C3.1 Background

A food delivery platform offering UPI as a payment method experienced an unusual pattern: customers were placing high-value orders, receiving delivery, and then claiming that their UPI payment "never went through." Investigation revealed that the platform's UPI integration had a race condition between the UPI collect request expiry and the order fulfilment flow.

C3.2 The Race Condition

The UPI collect flow works as follows: the merchant initiates a collect request, the customer receives a notification on their UPI app, and the customer has 5 minutes to approve or decline. The food delivery platform's order management system, however, started preparing the order immediately upon UPI collect initiation (not upon payment confirmation) to minimise delivery time.

Exploitative customers discovered that if they waited until the 4th minute to decline the UPI collect, the food was already prepared and dispatched. The platform's system marked the order as "payment pending" but had no automated cancellation flow — the delivery partner had already picked up the food. Customer support manually resolved each case, typically by absorbing the loss.

C3.3 Design Patterns for Prevention

- **Payment Confirmation Gate:** Order fulfilment should only begin AFTER the transaction reaches CAPTURED state (not AUTH_INITIATED).
- **UPI-Specific State Handling:** UPI collect requests must track the mandate window expiry. The state machine should automatically transition to AUTH_EXPIRED after the 5-minute window.
- **Webhook-Driven Fulfilment:** The order management system should subscribe to payment state changes and only trigger fulfilment on CAPTURED state.

SECTION C4: THE WEBHOOK REPLAY FRAUD

C4.1 Background

An online marketplace discovered that a sophisticated attacker had stolen ₹12.8 Lakh over three months by exploiting a webhook signature verification vulnerability. The attacker intercepted legitimate webhook payloads from the gateway and replayed them with modified amounts.

C4.2 The Attack Vector

The marketplace's webhook handler verified the Razorpay signature correctly. However, after signature verification, the handler extracted the payment amount from the webhook payload and used it to mark the order as paid - without cross-referencing the amount against the order's expected amount in the database.

The attacker's approach: (1) Place a high-value order (₹50,000); (2) Actually pay ₹10 to the gateway for an unrelated micro-transaction; (3) Intercept the legitimate ₹10 webhook; (4) Replay the webhook to the marketplace endpoint; (5) The handler sees a valid Razorpay signature for a ₹10 payment and marks the ₹50,000 order as "paid" without checking the amount.

C4.3 Critical Verification Steps

Your webhook handler must perform all of the following verifications after signature validation:

- **Amount Match:** The webhook amount must match the transaction's expected amount in the database.
- **Transaction ID Match:** The gateway's order/payment ID in the webhook must match the gateway reference stored during initiation.
- **Currency Match:** The webhook currency must match the transaction's currency.
- **Status Validity:** The webhook's reported status must be a valid next state from the transaction's current state.
- **Replay Detection:** The event ID must not exist in the processed_webhook_events table.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION C5: THE FLASH SALE CONNECTION POOL STAMPEDE

C5.1 Background

An Indian fashion e-commerce platform running its annual Republic Day sale experienced a complete payment system outage at 12:01 AM on 26th January - exactly when the sale went live. The platform had invested heavily in front-end scaling (CDN, auto-scaling web servers) but had overlooked the payment back-end's database connection pool limits.

C5.2 The Cascade Failure

At 12:00 AM, the platform's marketing campaign drove 2.5 million concurrent users to the site. Within 60 seconds, 180,000 checkout attempts hit the payment service simultaneously. The payment service used a PostgreSQL connection pool with a maximum of 20 connections. Each payment request held a database connection for the entire transaction lifecycle (including the 2–5 second gateway API call), exhausting the pool within seconds.

New payment requests queued for a database connection. The queue grew faster than connections were released, leading to connection timeout errors. The error handler for connection timeouts logged the error and - critically - attempted to write the error to the database, requiring another connection that did not exist. This created a recursive failure loop.

Time	Event	DB Connections	Success Rate
12:00:00	Sale goes live	20/20 (normal)	98.2%
12:00:15	Traffic spike begins	20/20 (saturated)	95.1%
12:00:30	Connection queue growing	20/20 + 500 queued	42.3%
12:00:45	Connection timeouts begin	20/20 + 2,800 queued	8.1%
12:01:00	Recursive error logging loop	20/20 + 12,000 queued	0.0%
12:01:15	PostgreSQL rejects connections	MAX hit	0.0%
12:01:30	Health check fails; service removed	N/A	Service down
12:15:00	Manual restart with increased pool	50/50	Recovering
12:25:00	Service stabilised	50/100 (normal)	96.8%

C5.3 Root Cause Analysis

- **Connection Holding During External Calls:** Database connections were held open while waiting for gateway API responses. A 3-second gateway call meant each connection was occupied for 3+ seconds, reducing effective throughput to ~6 transactions/second.
- **No Connection Pool Monitoring:** There was no alerting on connection pool utilisation. The team had no visibility into pool saturation until the outage occurred.
- **Synchronous Error Logging:** Error handling attempted synchronous database writes, creating a recursive failure when the database was already unavailable.
- **No Internal Circuit Breaker:** Circuit breakers were implemented for external gateways but not for internal resources (database, cache).

C5.4 Design Patterns for Prevention

- **Short-Lived Connections:** Never hold a database connection while waiting for an external API call. Acquire → read state → release → call gateway → acquire → write result → release.
- **PgBouncer for Connection Multiplexing:** PgBouncer in transaction pooling mode can serve hundreds of application connections through a smaller number of actual PostgreSQL connections.
- **Async Error Logging:** Error logs should be written to an in-memory buffer or file, never synchronously to the database experiencing the failure.
- **Internal Circuit Breakers:** When the database connection pool is saturated, fail fast with 503 Service Unavailable rather than queuing indefinitely.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION C6: DELIBERATE ERRORS FOR DETECTION

This project document contains 5 deliberate factual errors embedded within the training material and case studies. Identifying these errors is part of the assessment. Students who identify and document at least 3 of the 5 errors (with correct explanations) in a file named docs/errors-found.md will receive 25 bonus points. Identifying all 5 earns 50 bonus points.

Hint: The errors are in technical details - incorrect protocol specifications, wrong mathematical formulas, inaccurate gateway-specific behaviours, or subtly wrong database design recommendations. They are designed to test whether you actually understand the material or are blindly copying it.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

PART D: 15-DAY PROJECT METHODOLOGY & DELIVERABLES

This section provides the day-by-day methodology for completing the project. Each day has specific deliverables that must be committed to your GitHub repository. The repository must show a clean commit history reflecting daily progress - a single bulk commit on day 15 suggests the work was not done incrementally and will be penalised.

Day 1-2 - Architecture & Database Design:

Tasks:

- Study Part A training material thoroughly (sections A1–A7)
- Study all four case studies in Part C
- Design the complete database schema (minimum 10 tables as specified in A6.1)
- Create the database migration scripts using your chosen migration tool (Alembic for Python, Knex/Prisma for Node.js, Flyway for Java)
- Write the transaction state machine specification document (docs/state-machine.md) defining all states, events, and valid transitions
- Draw the state machine diagram using Mermaid syntax (embed in state-machine.md)
- Create the initial architecture document (docs/architecture.md) covering system components, data flow, and technology choices
- Write your first Architecture Decision Record (ADR) explaining your language/framework choice

Deliverable: Database migration scripts that execute cleanly; state-machine.md with Mermaid diagram; architecture.md first draft; ADR-001 language choice; Clean compile/run of empty project skeleton.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Day 3-4 - Transaction State Machine & Core Models:

Tasks:

- Implement the TransactionStateMachine class/module with all 12+ states
- Implement state transition validation (reject invalid transitions with clear error messages)
- Implement the Transaction model with all required fields
- Implement the TransactionStateLog model (immutable audit trail)
- Write unit tests for every valid state transition (minimum 20 test cases)
- Write unit tests for every invalid state transition (minimum 15 test cases)

- Implement the PII sanitisation function for gateway response logging
- Test that financial amounts use NUMERIC/BIGINT (not FLOAT)

Deliverable: TransactionStateMachine with 100% transition coverage; Transaction and TransactionStateLog models; 35+ unit tests all passing; PII sanitisation function with tests.

Day 5-6: Gateway Adapter Layer:

Tasks:

- Design and implement the PaymentGateway interface/abstract class
- Implement the RazorpayAdapter (mock - simulate gateway responses)
- Implement the StripeAdapter (mock)
- Implement the PayUAdapter (mock)
- Implement the UPIAdapter (mock - with collect flow simulation)
- Each adapter must correctly simulate: success, timeout, 5xx error, decline, and rate limit responses
- Implement configurable response delays to simulate real-world latency
- Write integration tests that verify each adapter produces correct state transitions

Deliverable: Four gateway adapters implementing the uniform interface; Each adapter simulates 5 response types; Integration tests verifying adapter-to-state-machine flow; ADR-002 explaining gateway simulation approach.

Day 7-8 - Routing Algorithm & Circuit Breaker

Tasks:

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

- Implement the GatewayRouter service with the multi-criteria scoring formula from A3.2
- Implement the CircuitBreaker class with CLOSED/OPEN/HALF-OPEN states
- Implement the GatewayHealthMetrics service (sliding window success rate, latency tracking)
- Seed the database with the historical performance data from A3.4
- Implement the routing configuration API (GET/PUT /api/v1/routing/config)
- Write unit tests for the scoring formula with known inputs and expected outputs
- Write unit tests for circuit breaker state transitions

- Implement the failover logic: when primary gateway fails, route to next-best within 2 seconds

Deliverable: GatewayRouter with configurable weights; CircuitBreaker with all three states; Health metrics tracking; Routing config API; 20+ unit tests for routing and circuit breaker.

Day 9-10 - Idempotency & Webhook Processing:

Tasks:

- Implement the IdempotencyService with the locking strategy from A4.1
- Implement the idempotency_keys database table with expiry and cleanup
- Implement webhook signature verification for all four gateways
- Implement the webhook deduplication layer using processed_webhook_events table
- Implement the webhook event processor that applies state machine transitions
- Implement out-of-order webhook handling (webhook arrives before API response)
- Write unit tests for idempotency (duplicate requests, concurrent requests, expired keys)
- Write integration tests for webhook processing (valid signature, invalid signature, duplicate event, amount mismatch)

Deliverable: IdempotencyService with locking; Webhook handlers for all 4 gateways; Deduplication layer; Webhook event processor; 25+ tests covering all webhook scenarios.

Day 11-12 - API Layer & Reconciliation Engine:

Tasks:

- Implement all 20+ API endpoints specified in A7.1
- Implement standard error response format from A7.2
- Implement request validation middleware (input sanitisation, schema validation)
- Implement authentication middleware (API key based)
- Implement the Reconciliation Engine batch job from A5.5
- Implement the anomaly detection and alerting for reconciliation mismatches
- Write the OpenAPI 3.0 specification (docs/api-specification.yaml)
- Write integration tests for each API endpoint (happy path + error cases)

Deliverable: All 20+ endpoints functional; OpenAPI spec; Reconciliation engine running on schedule; Request validation middleware; 30+ API integration tests.

Day 13 - Failure Scenario Testing:

Tasks:

- Write end-to-end tests for all 15 failure scenarios from Part B
- Each scenario test must set up the preconditions, trigger the failure, and assert the expected system behaviour
- Fix any bugs discovered during scenario testing
- Ensure all 15 scenarios pass without data corruption
- Run the full test suite and ensure minimum 80% code coverage
- Write a test coverage report documenting which code paths are covered and which are not

Deliverable: 15 end-to-end scenario tests all passing; Test coverage report showing 80%+ coverage; All bugs from scenario testing fixed and documented.

Day 14 - Docker, Documentation & Performance:

Tasks:

- Create the Dockerfile for the application
- Create docker-compose.yml including PostgreSQL, Redis (if used), and the application
- Verify that docker-compose up starts the entire system from scratch
- Write the comprehensive README.md with setup instructions, architecture overview, and API quick reference
- Complete the architecture.md with final design decisions
- Write the routing-algorithm.md documenting the algorithm specification
- Run performance benchmarks against the targets in B3
- Optimise any endpoints that fail to meet performance targets

Deliverable: Docker Compose setup that works with single command; Complete README.md; All documentation finalised; Performance benchmark results documented.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Day 15 - Final Review, Cleanup & Submission:

Tasks:

- Code cleanup: remove dead code, commented-out blocks, debug statements
- Ensure consistent code formatting (linter configured and passing)
- Run the complete test suite one final time
- Verify Docker deployment works from a clean state
- Review all documentation for accuracy and completeness
- Write a CHANGELOG.md summarising what was built
- Transfer the repository to the ZethetaIntern GitHub organisation
- Send confirmation email to hr@zetheta.com with repository link

Deliverable: Clean, well-formatted codebase; All tests passing; Docker deployment verified; Repository transferred to ZethetaIntern; Confirmation email sent.

Submission Instructions:

- All files to be named using convention: ProjectCode_YourName_DeliverableName (e.g., 201169C_RahulShah_MainReport)
- All PDFs must be readable and not password-protected
- Supporting Excel files must be in .xlsx format
- All daily deliverables across Days 1–15 are to be submitted on **Day 15** as a consolidated final submission. Students should combine their work into **one or two PDFs and/or one Excel file** wherever possible, rather than submitting separate files for each day. Clearly label each section within the consolidated document by its respective day and topic for ease of assessment.

Late submissions attract the penalties specified in Part F Assessment Criteria

PART E: NO-CODE TOOLS & PLATFORMS GUIDE

This section provides guidance on the recommended technology stack, development tools, and learning resources for implementing the payment orchestration layer. Students may choose any language/framework from the approved list below, but must justify their choice in an Architecture Decision Record (ADR).

SECTION E1: APPROVED TECHNOLOGY STACKS

Component	Option 1 (Python)	Option 2 (Node.js/TypeScript)	Option 3 (Java/Kotlin)
Web Framework	FastAPI (recommended)	Express.js or Fastify	Spring Boot
ORM / Database	SQLAlchemy + Alembic	Prisma or Knex.js	JPA / Hibernate + Flyway
Database	PostgreSQL 15+	PostgreSQL 15+	PostgreSQL 15+
Testing	pytest + pytest-asyncio	Jest + Supertest	JUnit 5 + Mockito
API Documentation	FastAPI auto-docs (Swagger UI)	swagger-jsdoc + swagger-ui-express	Springdoc OpenAPI
Task Queue (optional)	Celery + Redis	BullMQ + Redis	Spring Scheduler or Quartz
Containerisation	Docker + docker-compose	Docker + docker-compose	Docker + docker-compose
Code Quality	Black + Ruff + mypy	ESLint + Prettier + tsc	Checkstyle + SpotBugs

ADD EXPERIENCE TO KNOWLEDGE

SECTION E2: DEVELOPMENT ENVIRONMENT SETUP

E2.1 Required Tools

- **Git:** Version control. Repository must be initialised on Day 1 with a .gitignore appropriate for your language.
- **Docker Desktop:** For containerised deployment. Install from docker.com. Minimum version 24.0.
- **PostgreSQL 15+:** Install locally for development, or use the Docker Compose PostgreSQL container.
- **Postman or Insomnia:** For manual API testing during development.
- **pgAdmin or DBeaver:** For database inspection and query testing.
- **VS Code or IntelliJ IDEA:** Recommended IDEs with appropriate language extensions.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION E3: DIAGRAMMING & DOCUMENTATION TOOLS

- **Draw.io (diagrams.net):** Free browser-based diagramming tool. Use for architecture diagrams and flowcharts. Export as PNG and embed in your markdown documentation.
- **dbdiagram.io:** Free database schema diagramming tool. Use DBML syntax to define your schema and generate an ER diagram. Include the DBML source in your repository.
- **Mermaid:** Markdown-embeddable diagram syntax. GitHub natively renders Mermaid diagrams in markdown files. Use for state machine diagrams and sequence diagrams.
- **Swagger Editor (editor.swagger.io):** Online OpenAPI 3.0 specification editor with live preview. Use to write and validate your api-specification.yaml.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

SECTION E4: LEARNING RESOURCES

E4.1 Payment Systems Engineering

- Razorpay Engineering Blog - Payment Orchestration Architecture (razorpay.com/blog)
- Stripe Documentation - Payment Intents API (stripe.com/docs/payments/payment-intents)
- NPCI UPI Developer Documentation (npci.org.in)
- Martin Fowler - Patterns of Enterprise Application Architecture (martinfowler.com)

E4.2 State Machine Design

- Statecharts.dev - Interactive introduction to state machines and statecharts
- XState Documentation (xstate.js.org) - Even if not using XState, the concepts apply universally
- AWS Step Functions documentation - For understanding distributed state machines

E4.3 Distributed Systems Patterns

- Martin Fowler - Circuit Breaker Pattern (martinfowler.com/bliki/CircuitBreaker.html)
- Microsoft Azure Architecture - Idempotency Patterns (learn.microsoft.com)
- Designing Data-Intensive Applications by Martin Kleppmann (Book) - Chapters 8 and 9 on distributed systems
- YouTube: Hussein Nasser - Payment Systems Design playlist

E4.4 Database Design for Financial Systems

- PostgreSQL Documentation - NUMERIC type and financial calculations (postgresql.org/docs)
- PCI-DSS v4.0 Quick Reference Guide (pcisecuritystandards.org)
- RBI Master Direction on Payment Aggregators and Payment Gateways (rbi.org.in)

PART F: SUBMISSION PROTOCOL

Projects are assessed using a comprehensive 1000-point rubric covering seven dimensions: Problem Understanding, Solution Quality, Research & Analysis, Presentation & Clarity, Innovation & Creativity, Feasibility & Practicality and CV alignment. Assessment is conducted through a combination of manual review and AI-powered evaluation.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

END OF PROJECT DOCUMENT